오픈소스 기반 반도체 칩 종합 설계

권효재

이번에는 학습노트의 Q11과 Q13, Q14의 과정을 확인하며 설계 검증을 진행해 보았다. 교수님께서 수업시간에 말씀하셨던 것들을 토대로 실습을 진행하면서 내가 한 것들, 그리고 새롭게 알게 된 것들을 나만의 단어로 잘 설명할 수 있는지 되돌아보는 시간을 가졌다.

Q14의 talks 디렉터리 내의 텍스트 파일들을 확인해보니 이제까지 학습노트에서 단계별로 바이브코딩 설계를 하면서 추가되고 참고되었던 내용 (systemC 테스트벤치 사용, 테스트벤치 그래픽처리에 SDL2 사용 등등)을 반영해서 처음부터 한번에 제미나이에게 프롬프트로 입력해서 설계를 하고 실제로 기능을 하는 칩으로 만들 수 있게 검증하는 내용이었다. Q13은 talks 디렉터리가 따로 없었지만, 흡사한 대화내용인 Q11에 해당하는 부분을 학습자료에 남겨주셔서 같이 여러 단계를 거쳐 수정된 Q11의 코드에 대한 답변과 Q14의 talk 파일을 함께 보았다. RTL 코드를 합성하면 넷리스트로 바뀌고, 이걸 우리 ETRI 공정에 맞는 0.5um 표준셀로 바꿔서 레이아웃(스키메틱)을 만든다. 이때 만들어진 표준셀들의 배치 배선되고 만들어진 면적이 내가 만들 칩의 면적을 초과하면 절대 안된다. 합성 단계에서 문제가 없었다고 하더라도 작동하지 않는 칩이 탄생이 된다. 나중에 비싼 공정단계를 거친 칩을 가지고 할 수 있는 건 아무것도 없는데 껍데기만 남은 상황을 절대 만들지 않기 위해서 가장 중요한 단계이다. 합성 단계의 셀 면적을 고려해서 검증하고 수정해서 사고를 방지하는 것이다. 수업시간에 강조하셨듯이 실제 공정에 들어갈 칩의 면적에 대량의 코드들이 들어가기 위해서는 이 수치가 절대적으로 안에 들어갈 수 있고, 들어 맞아야 한다. 그래서 Q11과 Q14에서는 “면적” 초과하지 않도록 검증하는 엄청 중요한 파트라는 생각이 들었다.

Q11에서 알 수 있는 부분은 ETRI의 공정은 5um, 셀면적은 1.9*1.9(mm제곱)이고 여기에 설계를 맞춰야한다. 심지어 셀면적에 딱 맞아도 안되고 일정 밀도 기준을 맞추고 I/O패드까지 고려해한다. 플립플롭은 면적 때문에 이 차지 비율이 높고, 화면 출력에 관련된 스프라이트와 폰트는 ROM에 저장되고 참조할 수 있어야 하는데, 이 부분이 ROM모듈을 사용하지 못하고 합성가능하게 선언했기 때문에 조합논리회로로 자동합성되어 면적을 많이 잡아먹는다고 한다.

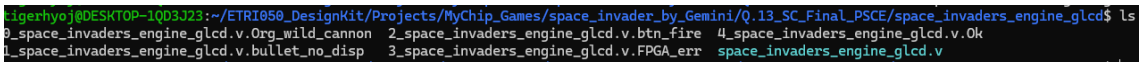
Q14에서도 Q11과 마찬가지로 플립플롭을 꼽았다. 면적 폭발 위험 요소로 중첩 반복문을 꼽았다. 이것이 Q11의 ROM 케이스와 같이, 합성되면 반복문으로 인해 조합논리회로들이 펼쳐져서 면적을 많이 잡아먹는다는 것이다. 또 조합논리회로중에서 곱셈기도 은근히 면적을 많이 차지해서 이걸 단순한 하나의 시프트 연산자로 쓰고, 카운터를 이용하면 면적이 많이 줄어든다고 한다. 코드를 조금 손봤더니 면적이 80%나 절감했다고 한다.

- simulation 디렉터리 내 make 실행 후 RTL 코드 비교

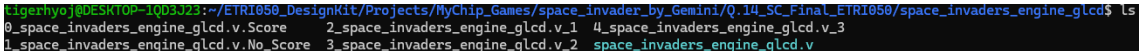
cat Makefile로 simulation 디렉토리 내에서 make를 실행한 결과, 따로 작성해두신 lint, build, run, wave, clean 목적 명령어들이 확인되었다.

링크된 것들이 어떤것이 있는지 확인하기 위해서 cat Makefile 메이크파일 내부를 확인해봤는데, # Build Rules 의 all 목적 명령어의 조건문에 "Try one of following RTLs,"라는 문구가 있는 걸 확인했고, 0~4까지 5개의 조건 별 실행 베릴로그 소스코드들이 있었다. 조건문을 살펴보니 심볼릭 링크 명령줄인

ln -s 와 ! -e "../$(TOP_MODULE)/$(TOP_MODULE).v" 를 확인할 수 있었는데, makefile 윗부분에서 TOP_MODULE이 TOP_MODULE = space_invaders_engine_glcd 로 문자열이 대입된걸 확인하고 상위폴더인 space_invaders_engine_glcd 내부의



Q13 심볼릭 링크 생성

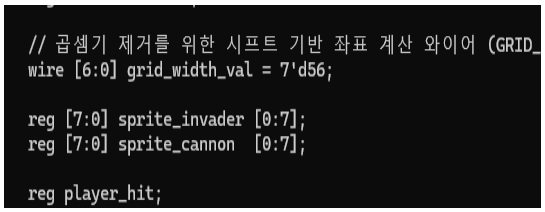


Q14 심볼릭 링크 생성

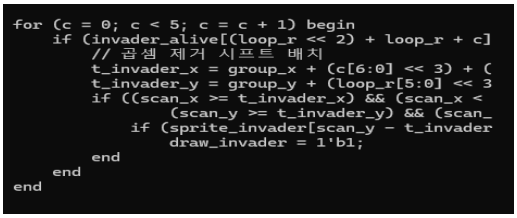
- space_invaders_engine_glcd.v 베릴로그 파일을 연결하고 있는걸 확인할 수 있었다. 어느 부분을 가르키고 있는지 확인했기 때문에, Q13,Q14의 space_invaders_engine_glcd 디렉터리로 직접 이동해서 코드들을 하나씩 확인해보았다.

Q13의 코드 0~4번 중 0번 RTL 코드를 살펴보았다. 1번 코드부터 살펴보니 거의 400줄이 조금 안되는 분량의 어마어마한 길이의 코드였다. Q13의 경우 각 (0)379줄 -> (1)414줄 -> (2)411줄 -> (3)401줄 -> (4)421줄이었다. Q13은 talks 디렉터리가 따로 없어서 바뀐결과를 찾기가 힘들었다. 그래서 가장 쉽게 볼 수 있는 앞단의 LCD 인터페이스 출력 버스 선언부 부분을 비교해보니, 2'b** -> 5'b***** 로 c언어의 const와 비슷한 localparam 자료형의 데이터가 5([4:0]) -> 2([1:0])로 줄였고, current_state, next_state가 저장된 reg 자료형 변수가 2비트 -> 5비트로 늘어났다, 또 group_x와 group_y reg 6,7비트 변수를 추가로 정의한 걸 확인할 수 있었다.

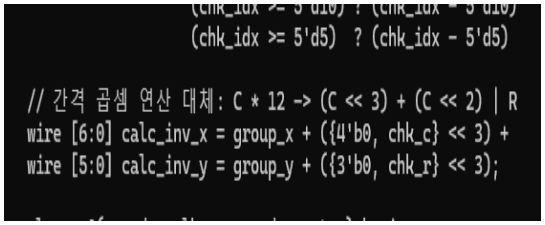
Q14는 talks의 프롬프트 답변을 통해 고쳤던 내용을 중심으로 탑다운 방식으로 변화된 코드들을 찾아나갔다. 앞서 언급했던 문제와 더불어 선언되어있지 않아 교수님이 c와 t_invader_x 및 t_invader_y를 추가했고 loop_r의 비트 범위를 개선한 것을 space_invaders_engine_glcd.v_3을 통해 확인할 수 있었고, 곱셈기를 시프트 연산자로 수정된 것, 중첩 반복문도 해결된 것을 확인할 수 있었다.



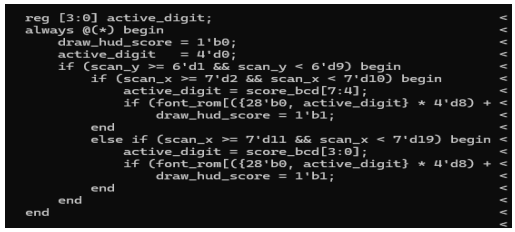
곱셈기 -> 시프트1



곱셈기 대체 시프트2



곱셈기 -> 시프트3



중첩반복문 (해결 전)

Q14도 Q13과 마찬가지로 수정된 코드가 훨씬 길이가 적었다. 코드 길이는 줄었는데, 합성됐을 때의 면적은 80%나 줄어든 것이다.

- Q14 emulation 디렉터리 내 FPGA를 위한 빌드

emulation의 make를 실행해보니 여러개의 make 목적 명령어들이 있었는데, 그중에서 빌드에 해당하는 [1. make build_mi 2. make build_trans 3. make build] 총 3개의 빌드를 진행했다. make 실행하면 실행되는 make all에 step이 잘 적혀있어서 진행하기 수월했다. 다만 build_mi 빌드 중간에는 fatal error: U8g2lib.h: No such file or directory 에러가 발생돼서 arduino-cli lib install "U8g2" 명령으로 라이브러리를 설치해서 해결했다.

추가적으로 make파일의 이해도를 높이기 위해서 makefile을 열어서 내부를 확인해보니 build: VERILOG_SRCS = \ $(RTL_PATH)/$(TOP_MODULE).v (윗부분에 TOP_MODULE = space_invaders_engine_glcd로 simulation때와 동일하게 매크로변수에 문자열을 넣었고, 컴파일 할 파일로 지정했다.) 를 확인할 수 있었는데, 앞서 simulation의 make파일에서 생성한 심볼릭 링크를 이용한 것이다. 만약 simulation 단계에서 make를 안했다면 emulation 단계에서 빌드도 안됐을 것이다.

Q13, Q14에 대해 emulation의 3개의 빌드를 끝냈는데, 1. make build_mi(모델링 인터페이스), 2. make build_trans(FPGA 트랜섹터), 3. make build(에뮬레이션 테스트벤치) 3개의 빌드를 각각 마쳤다.

하나의 프로젝트에 디렉터리까지 별도로 있는 중요한 시뮬레이션과 에뮬레이션에 대한 생각을 간단하게 정리하면 둘 다 반드시 칩이 만들어지기 전에 거쳐야 하는 검증 과정이다. 이제까지 하드웨어 소스코드에 테스트벤치라는 케이스를 넣어서 2진 디지털 파형을 보고, 우주침략게임의 경우 디지털 파형만으로 부족해서 화면출력을 통해 시뮬레이션을 해봤고, 앞으로 진행할 에뮬레이션은 FPGA라는 하드웨어 가상의 칩 위에서 실제 잘 동작을 하는지 검증하는 단계이다. 이 단계에서 논리적 오류를 병렬적인 하드웨어 동작을 통해 빠르고 확실하게 잡아줄 수 있다. 코드에 모순이 있어도 현실의 물리적인 요인을 담지 못하지만 베릴로그로 논리수준에서 시뮬레이션하여 검증을 할 수 있고 이걸 FPGA에서 에뮬레이션하는 것이다.

- Makefile에 대한 이해와 정리

개발자의 필수 도구인 make의 경우 make의 목적파일에 접근하고 실행하는 내부 명령줄들이 makefile 을 만든 사람마다 다르기 때문에, 우선 makefile 안의 목표와 변수들을 잘 파악해서 사용하는것이 최우선이라는 생각이 들었다. 앞선 파일 확인에도 단순하게 run이나 build 등등을 하는 경우에는 make 만 실행해서 기본 목표인 all이 실행돼서 all에 @echo로 적힌 내용대로 목표를 실행하면 되지만, 심볼릭 링크가 원본 디렉터리에 생성돼서 소스코드를 확인하기 위해선 makefile 내부 확인을 cat 으로 진행하는 과정이 필요(어느것을 가리키는지 확인하기 위해서)했던 경우가 좋은 예시인 것 같다. 작업을 자동화 해줄 수 있는 도구로 잘 사용하면 엄청 편하고, 앞으로 개발 명령어로 많이 활용해야 하는데, 그만큼 내부 구조를 한번에 파악하는데에 익숙해지고, 나도 커맨드라인 환경에서 쓸수있는 다른 make파일을 혼자 만들어 보면 좋겠다는 생각이 들었다.